

YOURSTORY



H2S



COCO CLI

HACKATHON

GCC Edition



FactoryPulse — IT/OT converged predictive maintenance with a local Granite agent

Team Name : ICE

Team Leader Name : Amogh Kotha Nagaraj

Team Size : 1

Problem Statement : **Predictive Maintenance and OEE Command Center**

Problem Brief: breakdowns hide in the OT/IT gap

Discrete manufacturing (machining, pressing, conveying, pumping) · Industry 4.0 IT/OT convergence

1 Real business problem

- Sensors show vibration and temperature drifting 24-48 h before a breakdown, but that data lives in OT historians.
- Production schedule, maintenance history, parts and cost live in ERP/CMMS. Nobody sees both at once.
- Result: failures are discovered when the machine stops, and OEE loss is explained weeks later.

2 Target users

- Reliability engineer / maintenance planner: who to inspect today, and why.
- Shift supervisor: what the line will deliver this shift and which asset is at risk.
- Plant manager: OEE by line and the cost of unplanned downtime.

3 Pain today → with FactoryPulse

- Found at breakdown → predicted ~20 h ahead, with drivers.
- Manual work order after the fact → P1 WO auto-created with evidence.
- Root cause = days across systems → grounded answer in seconds.
- OEE reported monthly → live by asset/line, tied to the cause.

4 Why now / domain context

- IT/OT convergence is the core of Industry 4.0 programmes; the data already exists, it is just siloed.
- Small local LLMs (Granite 4.0 1B) now run on a plant PC: no cloud round-trip for sensitive OT data.
- Snowflake Dynamic Tables, Tasks and Cortex Analyst give the same design a governed cloud twin.

Architecture: one pipeline from sensor stream to work order

Local MVP: SQLite · scikit-learn · Streamlit · Granite 4.0 1B GGUF | Snowflake twin: Dynamic Tables · Model Registry · Task · Cortex Analyst

DATA SOURCES

OT · sensor stream

vibration, temp, RPM, status
5-min samples · structured

IT · ERP production orders

planned/run min, counts, scrap
per shift · structured

IT · CMMS work orders

type, parts, cost, technician
free-text findings · unstructured

CONVERGED FEATURES

Hourly feature table

rolling stats, trends, baselines
+ hours since maintenance (IT)

OEE daily

Availability × Perf. × Quality
Snowflake: Dynamic Table

Evidence pack builder

sensor deltas, drivers, WO history
production context, for grounding

PREDICT & ACT

Failure model (GBM)

P(failure in 24 h) + top drivers
holdout-tested · Model Registry

Alert engine + rule limits

risk ≥ 0.7 CRITICAL · ≥ 0.4
WARNING
dedupe, auto-resolve, safety net

Work-order automation

P1 predictive WO with evidence
one WO per asset · 15-min Task

EXPERIENCE

Streamlit command center

overview, triage, asset explorer,
work orders, system health

Ask in natural language

verified queries → guarded SQL
read-only, auto-LIMIT, retry

Root cause · Granite 1B

grounded on the evidence pack
rule-based fallback if LLM is down

CoCo CLI skills used, and how they connect

Synthetic data generation: referentially consistent
OT + ERP + CMMS sets, no production data

Streamlit app generation: command center
scaffolded and iterated in CoCo

Data pipeline creation: hourly features & OEE as
Dynamic Tables; scheduled scoring Task

Automation + guardrails: unattended runs, read-
only SQL guard, alert dedupe, LLM fallback

Semantic model authoring: Cortex Analyst YAML
with verified queries (same tier as the app)

Reusable skill: coco/skills/factorypulse-pipeline:
build, run, validate, deploy

Impact: measurable now, scalable, extensible later

Figures from the generated plant (8 assets, 30 days, 17 unplanned failures, forward-looking holdout on the last 30 % of the timeline)

0.98

ROC-AUC on holdout

91%

of failure hours flagged in
the 24 h window (recall)

~20 h

average warning lead time
before breakdown

83 h

unplanned downtime in 30
days (~5 h per event)

< 1 min

from CRITICAL alert to P1
work order (was hours)

1 Measurable outcomes

- Precision 87% / recall 91%: few false alarms, most failures caught a shift ahead.
- Every CRITICAL alert carries its drivers and opens a work order with evidence, cutting triage from hours to seconds.
- Root-cause narrative in ~10 s and data questions in ~3 s on a CPU-only plant PC, grounded and audited.
- If 9 of 10 failures become planned interventions, most of the 83 h/month of unplanned downtime turns into shorter, scheduled work.

2 Scalability potential

- Snowflake twin: Dynamic Tables refresh incrementally, a Task scores every 15 min, the model sits in the Model Registry.
- Adding assets, lines or plants adds rows, not code: features, OEE and alerts are all per-asset.
- Cortex Analyst semantic model exposes the same verified queries in Snowsight agents and the Slackbot.
- Sensitive OT data can stay on site with the local Granite model; cloud handles scale and governance.

3 Beyond the demo

- Replace synthetic feeds with Snowpipe Streaming / Kafka for historians and ERP connectors for SAP.
- MCP connectors to the CMMS (SAP PM, Maximo) and Slack so the agent creates and routes real work orders.
- Feedback loop: closed work orders with technician verdicts retrain the model and refine failure modes.
- Survival model for time-to-failure, spare-parts forecasting, and a triage → RCA → planner multi-agent flow.

CoCo across the lifecycle, guardrails, and the 3-minute demo

1 Planning

coco/PLAN.md: problem framing, ontology, workflow, design decisions, validation plan drafted before the build

2 Development

pipeline, model, Streamlit app, Snowflake DDL / Dynamic Tables / semantic model built and iterated in CoCo

3 Execution

run_pipeline.py end-to-end; simulate_stream.py near-real-time ticks; scheduled Snowflake Task for unattended scoring

4 Testing

20 pytest checks: data integrity, OEE math, model floor, alert guardrails, SQL guard, LLM-free fallback

G Guardrails and graceful fallback

- Natural-language path can never write: single SELECT only, forbidden keywords blocked, auto-LIMIT, read-only connection.
- Verified queries answer common questions deterministically before the LLM is involved.
- Alerts de-duplicated per asset/type; only CRITICAL opens a work order; one open WO per asset; auto-resolve on recovery.
- Hard sensor limits (7.1 mm/s, 85 °C) alert even if the model is missing; LLM outage falls back to rule-based narrative.
- The LLM only sees the evidence pack, which is displayed next to every answer for audit.

D Demo flow

- Overview opens with CNC-002 and PMP-001 at CRITICAL, two auto-created P1 work orders, plant OEE by line.
- Alert triage: acknowledge, open the 72 h sensor view, ask Granite for the root cause (bearing wear signature).
- Investigate: "Which asset had the most unplanned downtime?" (verified) and a free-form question (text-to-SQL).
- Second terminal: simulate_stream.py --degrade CNC-003 raises a live alert and work order within a minute.

Repo: github.com/Amogh892/factorypulse-oe-command-center

YOURSTORY



1125

Coco CLI

HACKATHON

GCC Edition

THANK YOU

